

08-08-2026

Prefix Sum:

```
1480: class Solution {
    public int[] runningSum(int[] nums) {
        for(int i=1;i<nums.length;i++){
            nums[i]=nums[i-1]+nums[i];
        }
        return nums;
    }
}
```

```
303: class NumArray {
    private int[] prefix;
    public NumArray(int[] nums) {
        prefix = new int[nums.length + 1];
        for (int i = 0; i < nums.length; i++) {
            prefix[i + 1] = prefix[i] + nums[i];
        }
    }
    public int sumRange(int left, int right) {
        return prefix[right + 1] - prefix[left];
    }
}
```

```
724: class Solution {
    public int pivotIndex(int[] nums) {
        int total = 0;
        for (int num : nums) {
            total += num;
        }
        int leftSum = 0;
        for (int i = 0; i < nums.length; i++) {
```

```

        int rightSum = total - leftSum - nums[i];
        if (leftSum == rightSum) {
            return i;
        }
        leftSum += nums[i];
    }
    return -1;
}
}

525: class Solution {
    public int findMaxLength(int[] nums) {
        int n = nums.length;
        Map<Integer, Integer> mp = new HashMap<>();
        int sum = 0;
        int subArrayLength = 0;
        for (int i = 0; i < n; i++) {
            sum += nums[i] == 0 ? -1 : 1;
            if (sum == 0) {
                subArrayLength = i + 1;
            } else if (mp.containsKey(sum)) {
                subArrayLength = Math.max(subArrayLength, i - mp.get(sum));
            } else {
                mp.put(sum, i);
            }
        }
        return subArrayLength;
    }
}

```

Binary Search:

```

704: class Solution {
    public int search(int[] nums, int target) {
        int left = 0;
        int right = nums.length - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) {
                return mid;
            }
            else if (nums[mid] < target) {
                left = mid + 1;
            }
            else {
                right = mid - 1;
            }
        }
        return -1;
    }
}

```

```

35: class Solution {
    public int searchInsert(int[] nums, int target) {
        int left = 0;
        int right = nums.length - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) {
                return mid;
            }
            else if (nums[mid] > target) {
                right = mid - 1;
            }
            else {

```

```

        left = mid + 1;
    }
}
return left;
}
}

278: public class Solution extends VersionControl {
    public int firstBadVersion(int n) {
        int l=0,h=n;
        while(l<=h){
            int mid = l+(h-l)/2;
            if(isBadVersion(mid)) h = mid-1;
            else l = mid+1;
        }
        return l;
    }
}

```